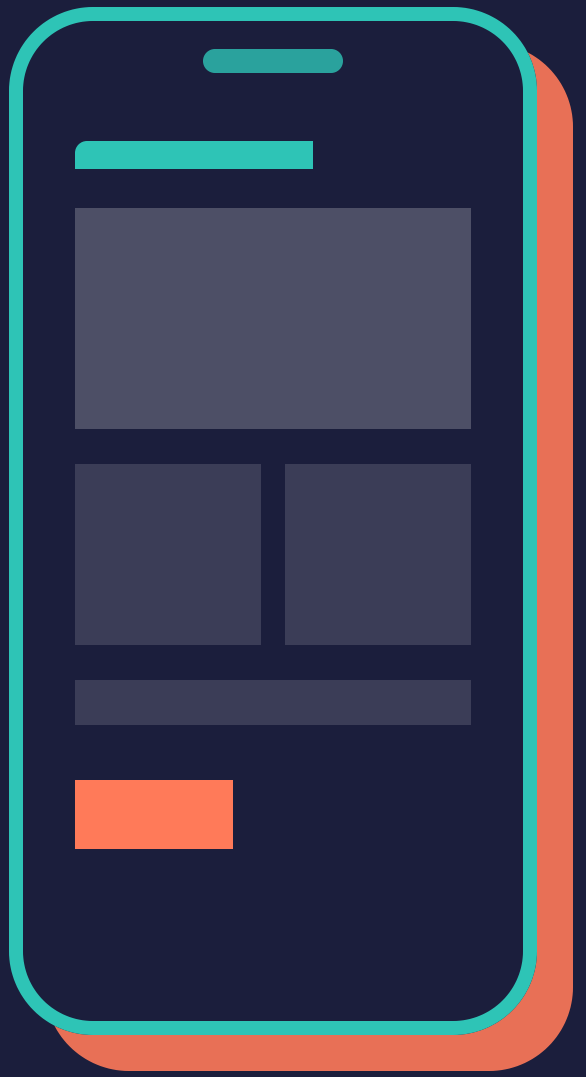
Architecture MVC et état

Cours 2 – Modèles, contrôleurs, vues et Context

Chris CHEVALIER



Sommaire

01 Pourquoi une architecture ?

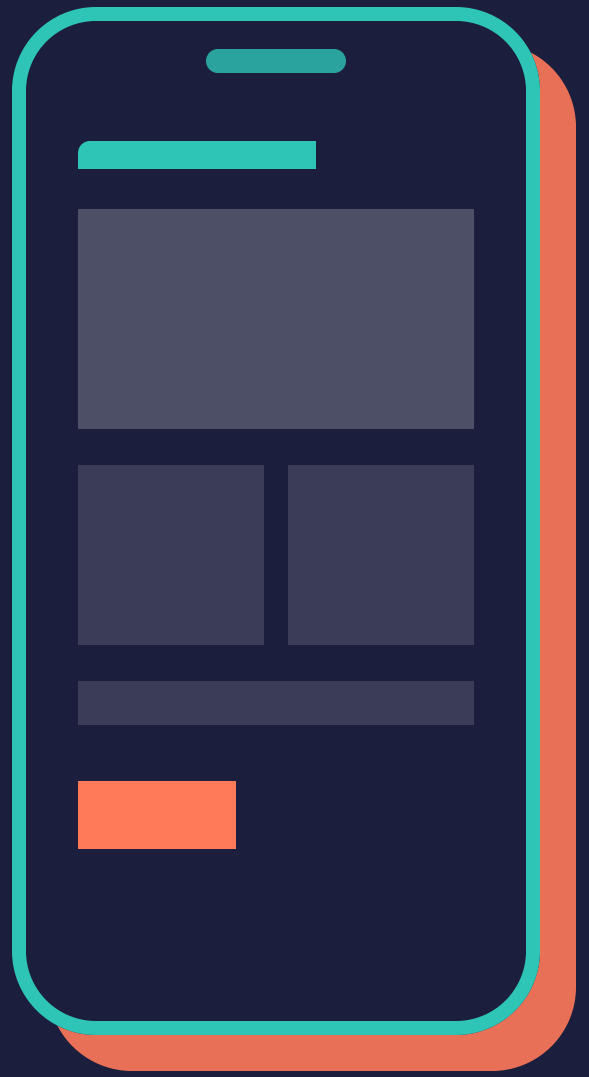
02 Le pattern MVC dans React

03 Le modèle : types et règles

04 Le contrôleur : services et hooks

05 La vue : des composants purs

06 L'état local et l'état global



01

Pourquoi une architecture ?

Le problème de l'écran « tout-en-un »



Un écran qui fait tout

src/app/avant.tsx (à ne pas suivre)

```tsx
export default function CatalogScreenBefore() {
  const [stays, setStays] = useState<Stay[]>([]);
  const [loading, setLoading] = useState(true);
  const [query, setQuery] = useState("");
  const [favorites, setFavorites] = useState<string[]>([]);
  useEffect(() => {
    setTimeout(() => { setStays(STAYS); setLoading(false); }, 600);
  }, []);
  if (loading) return <ActivityIndicator />;
  return (
    <FlatList
      data={stays.filter((s) => s.title.includes(query))}
      renderItem={({ item }) => (
        <Pressable onPress={() => setFavorites(favorites.includes(item.id)
          ? favorites.filter((f) => f !== item.id) : [...favorites, item.id])}>
          <Text>{item.title} · {item.pricePerNight} €</Text>
        </Pressable>
      )}
    />
  );
}
```

- Le chargement, les règles de recherche, les favoris et l'affichage sont **mélangés**
- Rien n'est **réutilisable** dans un autre écran
- Rien n'est **testable** sans afficher l'écran
- Remplacer les données locales par une API oblige à **réécrire l'écran**

Une responsabilité par fichier

> **Principe de responsabilité unique** (le « S » de SOLID) : un module ne doit avoir **qu'une seule raison de changer**.

Les données changent

On passe des données locales à une API, puis à Firebase. Seul le **service** change

Une règle change

La recherche doit aussi porter sur la ville. Seul le **modèle** change

Le design change

Les cartes deviennent des lignes. Seule la **vue** change

02

Le pattern MVC dans React

Modèle, vue, contrôleur



MVC : trois rôles

Rôle	Ce qu'il fait	Dans notre projet
Modèle	Décrit les données et les règles métier, sans rien afficher	`src/models/` : types et fonctions pures `src/services/` : accès aux données
Contrôleur	Tient l'état, réagit aux actions de l'utilisateur, appelle les services	`src/hooks/` : hooks personnalisés `src/context/` : état partagé
Vue	Affiche ce qu'on lui donne et signale les actions	`src/components/` : composants purs `src/app/` : écrans qui assemblent

> MVC vient des interfaces graphiques des années 1970. Dans React, le contrôleur n'est pas une classe : c'est un **hook personnalisé**.

Le trajet d'une action

1. Vue

L'utilisateur tape « Lyon ». `StayFiltersBar` appelle `onChange({ query: "Lyon" })`

2. Contrôleur

`useStayFilters` met à jour son état `filters`

3. Modèle

`matchesFilters()` décide quels séjours correspondent

4. Vue

React affiche à nouveau la liste avec les séjours visibles

> Les données **descendent** par les props, les événements **remontent** par les fonctions de rappel. La vue ne modifie jamais les données elle-même.

L'arborescence cible

src/

```
src/
├── app/                    VUE : écrans
│   ├── _layout.tsx         fournit le Context
│   ├── index.tsx           catalogue
│   └── favorites.tsx       favoris
├── components/             VUE : composants purs
│   ├── stay-card.tsx
│   ├── stay-filters-bar.tsx
│   └── state-message.tsx
├── hooks/                  CONTRÔLEUR
│   ├── use-stays.ts
│   └── use-stay-filters.ts
├── context/                CONTRÔLEUR partagé
│   └── favorites-context.tsx
├── services/               MODÈLE : accès aux données
│   └── stays-service.ts
├── models/                 MODÈLE : types et règles
│   └── stay.ts
└── data/stays.ts           données locales
```

- Un fichier par **domaine** : les séjours, les favoris…
- Noms de fichiers en **kebab-case** (convention d'Expo), composants en PascalCase, hooks en `useXxx`
- C'est la structure attendue pour le projet

03

Le modèle : types et règles

Des données décrites et des fonctions pures



Modéliser les données

src/models/stay.ts (début)

```ts
export type StayType = "apartment" | "house" | "cabin";

export type Stay = {
  id: string; title: string; city: string; type: StayType;
  pricePerNight: number; available: boolean; imageUrl?: string;
};

export type StayFilters = {
  query: string;
  type: StayType | null;
  onlyAvailable: boolean;
};

export const STAY_TYPE_LABELS: Record<StayType, string> = {
  apartment: "Appartement",
  house: "Maison",
  cabin: "Cabane",
};
```

- Les types sont le **contrat** entre toutes les couches
- `StayType | null` : « aucun type choisi » est une valeur prévue
- `Record<StayType, string>` : un objet avec **une entrée par type**. En ajouter un sans libellé est une erreur

Les règles métier : des fonctions pures

src/models/stay.ts (suite)

```ts
// Règle métier : une fonction pure, sans React
export function matchesFilters(
  stay: Stay,
  filters: StayFilters
): boolean {
  const search = filters.query.trim().toLowerCase();
  const matchesQuery =
    stay.title.toLowerCase().includes(search) ||
    stay.city.toLowerCase().includes(search);
  const matchesType =
    filters.type === null || stay.type === filters.type;
  const matchesAvailability =
    !filters.onlyAvailable || stay.available;
  return matchesQuery && matchesType && matchesAvailability;
}
```

- **Pure** : le même résultat pour les mêmes paramètres, sans effet de bord
- Elle ne connaît ni React ni l'écran : on peut la **tester** en une ligne et la **réutiliser** partout
- Une règle change ? On modifie cette fonction, et seulement elle

04

Le contrôleur : services et hooks

Charger, filtrer, réagir



Le service : un contrat d'accès aux données

src/services/stays-service.ts

```ts
import { STAYS } from "@/data/stays";
import { Stay } from "@/models/stay";

const wait = (ms: number) =>
  new Promise((resolve) => setTimeout(resolve, ms));

// Aujourd'hui des données locales,
// demain une API ou Firebase
export async function fetchStays(): Promise<Stay[]> {
  await wait(600);
  return STAYS;
}
```

- Le service promet une chose : `Promise<Stay[]>`
- Le délai simule le réseau : l'écran doit déjà gérer le **chargement**
- Au TP3, son contenu deviendra un `fetch()`. Au TP4, un appel à Firebase. **Ni les hooks ni les vues ne changeront**

Un état qui ne peut pas mentir

src/hooks/use-stays.ts (début)

```ts
export type StaysState =
  | { status: "loading" }
  | { status: "error"; message: string }
  | { status: "success"; stays: Stay[] };
```

- Une **union discriminée** : la propriété `status` indique la forme de l'objet
- Impossible d'avoir à la fois une erreur et des données. Avec trois booléens (`loading`, `error`, `data`), huit combinaisons existent, dont cinq absurdes

Dans l'écran, TypeScript suit le raisonnement

```ts
if (state.status === "error") return <StateMessage message={state.message} onRetry={reload} />;
// ici, TypeScript sait que state.status vaut "success" ou "loading"
```

Le hook contrôleur `useStays`

src/hooks/use-stays.ts (suite)

```ts
export function useStays() {
  const [state, setState] = useState<StaysState>({ status: "loading" });
  const [attempt, setAttempt] = useState(0);
  useEffect(() => {
    let cancelled = false;
    fetchStays()
      .then((stays) => { if (!cancelled) setState({ status: "success", stays }); })
      .catch(() => {
        if (!cancelled)
          setState({ status: "error", message: "Échec du chargement" });
      });
    return () => { cancelled = true; };
  }, [attempt]);
  const reload = () => {
    setState({ status: "loading" });
    setAttempt((n) => n + 1);
  };

  return { state, reload };
}
```

- Un **hook personnalisé** est une fonction `useXxx` qui utilise d'autres hooks
- `cancelled` évite de modifier l'état d'un écran déjà quitté
- Changer `attempt` relance l'effet : c'est le bouton « Réessayer »
- Le hook renvoie **ce dont la vue a besoin**, rien de plus

Le hook `useStayFilters` : un état dérivé

src/hooks/use-stay-filters.ts

```ts
import { useState } from "react";
import { matchesFilters, Stay, StayFilters } from "@/models/stay";

const INITIAL_FILTERS: StayFilters = {
  query: "", type: null, onlyAvailable: false,
};

export function useStayFilters(stays: Stay[]) {
  const [filters, setFilters] =
    useState<StayFilters>(INITIAL_FILTERS);

  const updateFilters = (changes: Partial<StayFilters>) =>
    setFilters((current) => ({ ...current, ...changes }));

  const visibleStays =
    stays.filter((stay) => matchesFilters(stay, filters));

  return { filters, updateFilters, visibleStays };
}
```

- `visibleStays` n'est **pas un état** : il se calcule à chaque rendu
- Stocker une donnée calculable crée deux sources de vérité qui finissent par diverger
- `Partial<StayFilters>` : toutes les propriétés deviennent facultatives
- Le React Compiler, activé par Expo, évite les calculs inutiles : pas besoin de `useMemo`

Quiz

Je dois trier les séjours par prix. Où écrire la fonction de tri ?

> Dans le modèle (`src/models/stay.ts`) : c'est une règle métier, pure et testable. Le hook l'appelle, la vue n'en sait rien.

Faut-il un `useState` pour le nombre de séjours visibles ?

> Non : `visibleStays.length` se calcule. Un état ne doit contenir que ce qui ne se déduit de rien d'autre.

05

La vue : des composants purs

Afficher et signaler, rien d'autre



Un composant d'affichage pur

src/components/stay-filters-bar.tsx (extrait)

```tsx
type StayFiltersBarProps = {
  filters: StayFilters;
  onChange: (changes: Partial<StayFilters>) => void;
};

export function StayFiltersBar({ filters, onChange }: StayFiltersBarProps) {
  return (
    <View style={styles.bar}>
      <TextInput
        value={filters.query}
        onChangeText={(query) => onChange({ query })}
      />
      <Switch
        value={filters.onlyAvailable}
        onValueChange={(onlyAvailable) => onChange({ onlyAvailable })}
      />
      {/* … boutons de type … */}
    </View>
  );
}
```

- Il **reçoit** les filtres et **signale** les changements
- Aucun état, aucun service, aucun `useEffect`
- On peut le réutiliser ailleurs ou changer son design sans toucher à la logique

L'écran assemble

src/app/index.tsx (extrait)

```tsx
export default function CatalogScreen() {
  const columns = useColumns();
  const { state, reload } = useStays();
  const stays = state.status === "success" ? state.stays : [];
  const { filters, updateFilters, visibleStays } = useStayFilters(stays);
  const { favorites, isFavorite, toggleFavorite } = useFavorites();
  if (state.status === "loading") return <StateMessage loading />;
  if (state.status === "error")
    return <StateMessage message={state.message} onRetry={reload} />;
  return (
    <View style={styles.container}>
      <Link href="/favorites">Mes favoris ({favorites.length})</Link>
      <StayFiltersBar filters={filters} onChange={updateFilters} />
      <FlatList key={columns} numColumns={columns} data={visibleStays}
        renderItem={({ item }) => (
          <StayCard stay={item} isFavorite={isFavorite(item.id)}
            onToggleFavorite={toggleFavorite} />
        )} />
    </View>
  );
}
```

- L'écran ne contient **aucune règle** : il branche les hooks sur les composants
- Il gère les trois états : chargement, erreur, succès
- Les hooks sont appelés **avant** tout `return` : c'est une règle de React

Check-list d'une vue pure

Une vue pure...

- reçoit ses données par des **props typées**
- signale les actions par des **fonctions de rappel** (`onChange`, `onPress`...)
- peut avoir un petit état **d'interface** (un menu ouvert, un champ en cours de saisie)

Une vue pure ne...

- n'importe **aucun service** et n'appelle ni `fetch` ni Firebase
- ne contient **aucune règle métier** (calcul de prix, filtre, tri)
- ne charge pas de données dans un `useEffect`

> C'est la grille que j'utiliserai pour évaluer l'architecture du projet : **séparation stricte des responsabilités**.

06

L'état local et l'état global

Partager une donnée entre plusieurs écrans



Local ou global ?

	État local	État global
Exemple	Le texte d'une recherche, un menu ouvert	Les favoris, l'utilisateur connecté, le panier
Utilisé par	Un écran et ses enfants directs	Plusieurs écrans éloignés
Outil	`useState` dans un composant ou un hook	`Context` , ou une bibliothèque

> Faire descendre une prop à travers cinq composants qui ne s'en servent pas s'appelle le **prop drilling**. C'est le signe qu'un état global est nécessaire.

Créer un Context typé

src/context/favorites-context.tsx (début)

```tsx
type FavoritesContextValue = {
  favorites: string[];
  isFavorite: (id: string) => boolean;
  toggleFavorite: (id: string) => void;
};

const FavoritesContext = createContext<FavoritesContextValue | null>(null);
export function FavoritesProvider({ children }: { children: ReactNode }) {
  const [favorites, setFavorites] = useState<string[]>([]);
  const isFavorite = (id: string) => favorites.includes(id);
  const toggleFavorite = (id: string) =>
    setFavorites((current) => current.includes(id)
      ? current.filter((f) => f !== id) : [...current, id]);
  return (
    <FavoritesContext.Provider value={{ favorites, isFavorite, toggleFavorite }}>
      {children}
    </FavoritesContext.Provider>
  );
}
```

- Le type décrit ce que le Context **met à disposition**
- `null` par défaut : la valeur n'existe que sous un Provider
- Le **Provider** tient l'état et le transmet à tous ses descendants

Lire le Context avec un hook

src/context/favorites-context.tsx (fin)

```tsx
export function useFavorites(): FavoritesContextValue {
  const context = useContext(FavoritesContext);
  if (!context) {
    throw new Error(
      "useFavorites doit être utilisé dans un FavoritesProvider"
    );
  }
  return context;
}
```

- Le hook `useFavorites()` est la seule porte d'entrée. Les écrans n'importent jamais le Context directement
- L'erreur explicite signale un oubli du Provider dès le premier test
- Après le `if`, TypeScript sait que `context` n'est plus `null`

Fournir le Context à toute l'application

- `_layout.tsx` enveloppe **tous les écrans** : c'est l'endroit du Provider
- Le catalogue ajoute un favori, l'écran `favorites.tsx` le voit aussitôt
- Chaque modification de la valeur affiche à nouveau **tous** les composants qui lisent ce Context. Ne mettez pas tout dedans

src/app/_layout.tsx

```tsx
import { Stack } from "expo-router";
import { FavoritesProvider } from "@/context/favorites-context";

export default function RootLayout() {
  return (
    <FavoritesProvider>
      <Stack />
    </FavoritesProvider>
  );
}
```

src/app/favorites.tsx (extrait)

```tsx
const { isFavorite } = useFavorites();
const favoriteStays =
  state.stays.filter((s) => isFavorite(s.id));
```

Et les bibliothèques d'état ?

Zustand

Un état global en quelques lignes, sans Provider. Seuls les composants concernés sont affichés à nouveau

Redux Toolkit

Le standard historique des grosses applications, plus verbeux, avec des outils de débogage poussés

TanStack Query

Spécialisé dans les **données du serveur** : cache, rechargement, état de chargement

Dans ce module, **Context suffit**. Une bibliothèque est un bonus du projet si vous savez expliquer pourquoi vous l'avez choisie.

Quiz

Un composant appelle `useFavorites()` mais l'application plante avec « useFavorites doit être utilisé dans un FavoritesProvider ». Pourquoi ?

> Le composant n'est pas à l'intérieur du `FavoritesProvider`. Il faut l'ajouter au-dessus, en général dans `_layout.tsx`.

Le texte d'un champ de recherche doit-il aller dans un Context ?

> Non. Il ne sert qu'à un écran : un état local suffit. Le Context est réservé à ce que plusieurs écrans partagent.

À retenir

- **Modèle** : des types et des fonctions pures dans `models/`, l'accès aux données dans `services/`
- **Contrôleur** : des hooks personnalisés qui tiennent l'état et appellent les services
- **Vue** : des composants qui reçoivent des props et signalent des événements
- Un état en **union discriminée** rend les états absurdes impossibles
- Ce qui se calcule ne se stocke pas
- **Context** pour l'état partagé entre écrans, exposé par un hook

Des questions ?

Chris CHEVALIER
chevalier@chris-freelance.com

